Міністерство освіти і науки України
Український державний університет науки і технологій
Навчально-науковий інститут
«Український державний хіміко-технологічний університет»
(повне найменування вищого навчального закладу)

Кафедра інформаційних систем
(повна назва кафедри)

КУРСОВА РОБОТА

з дисципліни «Проєктування інформаційних систем»
(назва дисципліни)

на тему: Проєктування інформаційної системи «Оптимізація розподілу вантажів у транспортній мережі»

Студента (ки) 4 курсу 4-КН-22 групи
спеціальності 122 Комп'ютерні науки

Ніколаєв Т.О.
(прізвище та ініціали)

Керівник _____
_____
(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна шкала _____

Кількість балів: _____ Оцінка: ECTS _____

Члени комісії _____ _____
(підпис) (прізвище та ініціали)

_____ _____
(підпис) (прізвище та ініціали)

м. Дніпро – 2026 рік

Міністерство освіти і науки України

Український державний університет науки і технологій

Навчально-науковий інститут
«Український державний хіміко-технологічний університет»
(повне найменування вищого навчального закладу)

Факультет, відділення комп'ютерних наук та інженерії, денне
Кафедра інформаційних систем
Спеціальність 122 Комп'ютерні науки
Курс 4 Група 4-КН-22 Семестр 8 Тетраместр 15

Завдання
на курсовий проект (роботу) студенту

Ніколаєв Тимур Олександрович
(прізвище, ім'я, по-батькові)

1. Тема проекту (роботи) Проєктування інформаційної системи «оптимізації розподілу вантажів у транспортній мережі»

2. Строк здачі студентом закінченого проекту (роботи) 03 квітня 2026 року

3. Вихідні дані до проекту (роботи) *Адреси точок доставки, параметри транспортних засобів, дані вуличної мережі, координати депо.*

4. Зміст розрахунково-пояснювальної записки (перелік питань, які підлягають розробці)
ВСТУП
1 ЗАГАЛЬНИЙ РОЗДІЛ
2 СПЕЦІАЛЬНИЙ РОЗДІЛ
3 РОЗРАХУНКОВИЙ РОЗДІЛ
ВИСНОВКИ
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

6. Дата видачі завдання 16 лютого 2026 року

КАЛЕНДАРНИЙ ПЛАН

№ п/п	Назва етапів курсового проекту (роботи)	Строк виконання етапів проекту (роботи)	Примітки
1	Отримання індивідуального завдання до курсової роботи	16.02.2026	
2	Аналіз предметної області	16.02.2026 – 18.02.2026	
3	Проєктування інформаційної системи	19.02.2026 – 27.02.2026	
4	Оформлення пояснювальної записки	28.02.2026 – 15.03.2026	
5	Представлення курсової роботи на перевірку	23.03.2026	
6	Захист курсової роботи	03.04.2026	

Студент _____ Ніколаєв Т.О.
(підпис) (прізвище та ініціали)

Керівник роботи _____ _____
(підпис) (прізвище та ініціали)

6 лютого 20 26 року

Реферат

Пояснювальна записка: 30 с., 10 рис., 10 джерел.

ПРОЄКТУВАННЯ, ІНФОРМАЦІЙНА СИСТЕМА, UML, ОПТИМІЗАЦІЯ МАРШРУТІВ, ЗАДАЧА КОМІВОЯЖЕРА, OPENSTREETMAP, МУЛЬТИМОДАЛЬНА МАРШРУТИЗАЦІЯ.

Об'єктом курсової роботи є оптимізація маршрутів доставки вантажів у міській транспортній мережі.

Предмет роботи – інформаційна система оптимізації розподілу вантажів у транспортній мережі.

Мета роботи – проєктування інформаційної системи оптимізації маршрутів доставки з використанням об'єктно-орієнтованого підходу та мови моделювання UML.

Завдання роботи – аналіз предметної області маршрутизації транспортних засобів, проєктування інформаційної системи з використанням UML-діаграм, опис реалізації ключових алгоритмів оптимізації.

Методи дослідження – об'єктно-орієнтоване моделювання.

Засоби розробки – засіб моделювання PlantUML та мова моделювання UML.

Структура роботи включає три розділи. Загальний розділ містить аналіз предметної області, огляд існуючих рішень та постановку задачі. В спеціальному розділі представлено проєктування інформаційної системи з використанням UML-діаграм. В розрахунковому розділі описано вибір засобів розробки, структуру програмного забезпечення та ключові алгоритми.

Робота має практичне значення та може бути використана при створенні відповідної інформаційної системи оптимізації маршрутів доставки.

ЗМІСТ

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАЧЕННЯ.....6
ВСТУП.....7
1 ЗАГАЛЬНИЙ РОЗДІЛ.....8
1.1 Опис предметної області.....8
1.2 Огляд існуючих рішень.....8
1.3 Постановка задачі.....9
2 СПЕЦІАЛЬНИЙ РОЗДІЛ.....11
2.1 Діаграма прецедентів (Use Case Diagram).....11
2.2 Діаграми класів (Class Diagram).....13
2.2.1 Діаграма класів предметної області.....13
2.2.2 Діаграма класів модульної структури.....15
2.3 Діаграми послідовності (Sequence Diagram).....17
2.3.1 Процес оптимізації маршруту (TSP).....17
2.3.2 Геокодування адреси доставки.....19
2.3.3 Оптимізація VRP (декілька транспортних засобів).....20
2.4 Діаграма діяльності (Activity Diagram).....22
2.5 Діаграма станів (State Diagram).....25
2.6 Діаграма компонентів (Component Diagram).....27
2.7 Діаграма розгортання (Deployment Diagram).....29
3 РОЗРАХУНКОВИЙ РОЗДІЛ.....30
3.1 Вибір засобів розробки.....30
3.2 Структура програмного забезпечення.....30
3.3 Ключові алгоритми.....31
ВИСНОВКИ.....33

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАЧЕННЯ

VRP — Vehicle Routing Problem. Задача маршрутизації транспортних засобів

CVRP — Capacitated Vehicle Routing Problem. Задача маршрутизації транспортних засобів з обмеженнями вантажопідйомності

TSP — Travelling Salesman Problem. Задача комівояжера

ВСТУП

Сучасна логістика стикається з постійно зростаючим обсягом доставок, що вимагає ефективних інструментів планування маршрутів. Задача маршрутизації транспортних засобів є однією з ключових задач комбінаторної оптимізації, що належить до класу NP-складних задач.

Метою даної курсової роботи є проєктування інформаційної системи для оптимізації розподілу вантажів у транспортній мережі з використанням об'єктно-орієнтованого підходу та мови моделювання UML. Система використовує реальні дані вуличної мережі OpenStreetMap та підтримує мультимодальну маршрутизацію (автомобіль, велосипед, пішки).

Об'єктом проєктування є інформаційна система оптимізації маршрутів доставки вантажів у міській транспортній мережі.

Предметом проєктування є архітектура, структура класів, взаємодія компонентів та поведінка системи.

Підхід до проєктування: об'єктно-орієнтований (ООП) з використанням уніфікованої мови моделювання UML.

1 ЗАГАЛЬНИЙ РОЗДІЛ

1.1 Опис предметної області

Предметна область — оптимізація маршрутів доставки вантажів у міській транспортній мережі. Транспортну мережу формалізовано як мультимодальний зважений орієнтований граф $G = (V, E, \Phi, \Psi)$, де:

– $V = \{v_1, v_2, \ldots, v_n\}$ — скінченна множина вузлів мережі (перехрестя, точки інтересу);

– $E \subseteq V \times V$ — множина орієнтованих ребер (дорожні сегменти);

– $\Phi : V \to A$ — функція атрибутів вузлів (координати, часові вікна);

– $\Psi : E \to B$ — функція атрибутів ребер (довжина, допустимі типи транспорту).

Система DeliveryIQ підтримує три типи транспортних засобів: $T = \{Car, Bike, Walk\}$, кожен з яких має характеристичну швидкість та вантажну місткість. Кожне ребро графа може бути доступним для одного або кількох типів транспорту відповідно до тегів OpenStreetMap (highway, motor_vehicle, bicycle, foot, oneway).

Задача оптимізації формулюється як задача змішаного цілочислового програмування — CVRP, де цільова функція мінімізує зважену суму загального часу доставки та кількості задіяних транспортних засобів.

1.2 Огляд існуючих рішень

На ринку існує ряд рішень для оптимізації маршрутів доставки:

– *Google OR-Tools* – потужний фреймворк від Google для комбінаторної оптимізації, але вимагає значного досвіду програмування та не має інтегрованого веб-інтерфейсу;

– *Route4Me, OptimoRoute* – комерційні SaaS-рішення з широким функціоналом, але з високою вартістю підписки та відсутністю контролю над алгоритмами оптимізації;

– *OSRM, Valhalla* – відкриті маршрутизатори на базі OSM, але без вбудованої підтримки VRP-оптимізації та мультимодальності в одному інтерфейсі.

Розроблювана система DeliveryIQ відрізняється від існуючих рішень тим, що поєднує: (1) роботу з реальними даними OSM; (2) мультимодальну маршрутизацію в єдиному інтерфейсі; (3) відкритий код та можливість розгортання на власному сервері; (4) гнучкий вибір алгоритмів оптимізації.

1.3 Постановка задачі

Необхідно спроєктувати інформаційну систему, яка забезпечує:

1) геокодування адрес доставки у географічні координати;

2) завантаження та обробку реальної вуличної мережі з OpenStreetMap;

3) побудову мультимодальних графів з урахуванням правил доступу для різних типів транспорту;

4) обчислення матриці відстаней (часів переїзду) за алгоритмом Дейкстри;

5) розв'язання задачі комівояжера (TSP) та задачі маршрутизації з обмеженнями на місткість (CVRP);

6) підтримку механізму «останньої милі» для автомобільного режиму;

7) візуалізацію оптимальних маршрутів на інтерактивній карті;

8) керування базою даних пакунків зі статусами доставки.

2 СПЕЦІАЛЬНИЙ РОЗДІЛ

Для проєктування інформаційної системи обрано об'єктно-орієнтований підхід з використанням уніфікованої мови моделювання UML (Unified Modeling Language). Нижче наведено комплект UML-діаграм, що описують статичну структуру, динамічну поведінку та фізичну архітектуру системи.

2.1 Діаграма прецедентів

Діаграма прецедентів визначає функціональні вимоги системи з точки зору її користувачів (акторів). Основним актором є Оператор логістики, який взаємодіє із системою через веб-інтерфейс.

Зовнішні актори-системи:

– Nominatim API — сервіс геокодування OpenStreetMap для перетворення адрес у координати;

– OpenStreetMap (Overpass API) — джерело даних вуличної мережі;

– Mapbox API — сервіс для отримання матриці часу з урахуванням трафіку.

Основні прецеденти включають:

– обрання міста доставки та задання адреси депо;

– додавання точок доставки;

– керування парком транспортних засобів (тип, місткість);

– запуск оптимізації маршруту (включає завантаження мережі, побудову матриці, розв'язання VRP);

– перегляд результатів на інтерактивній карті та у табличному вигляді;

– керування базою пакунків.

Діаграма прецедентів представлена на рисунку 2.1.

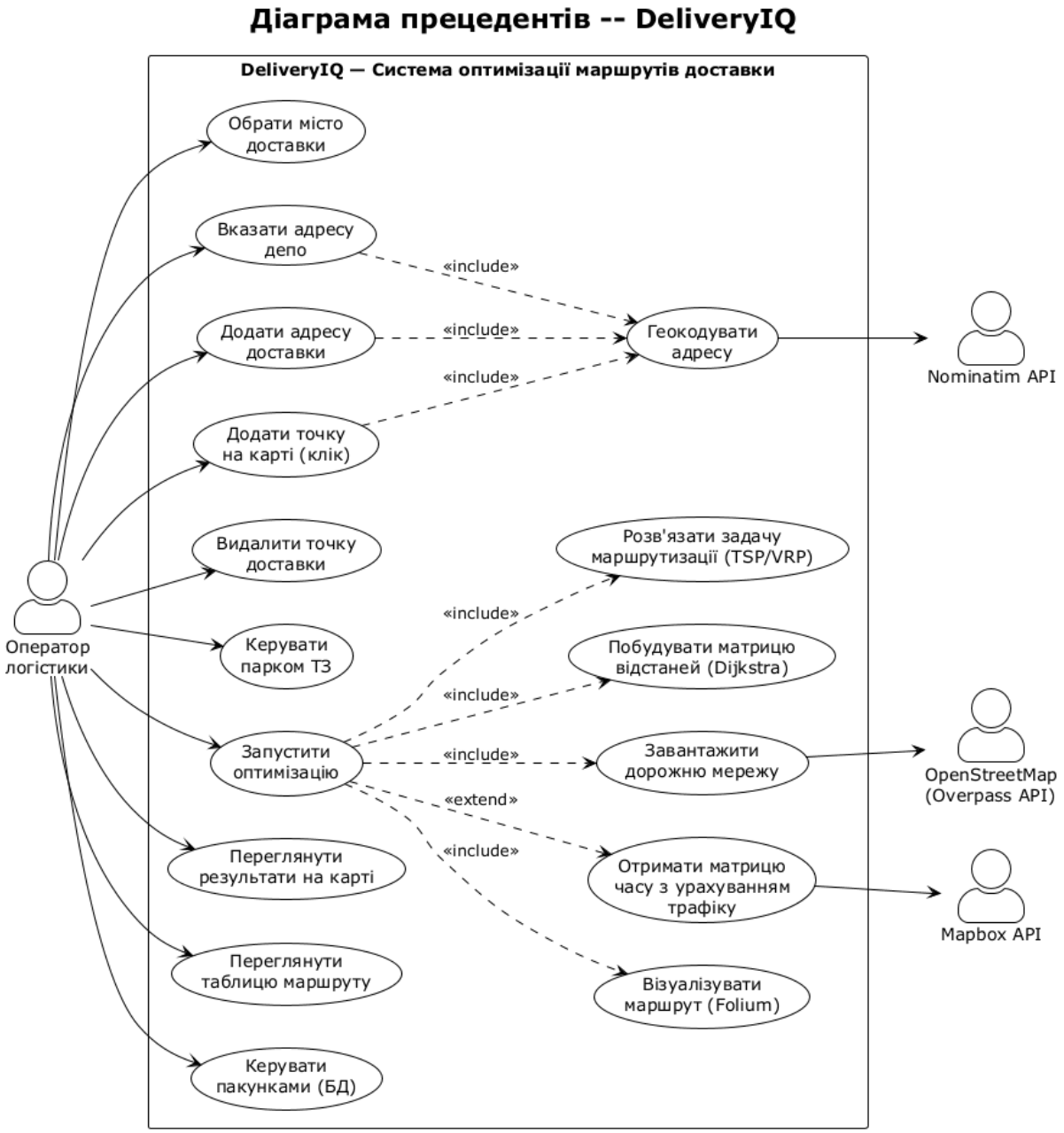

Рисунок 2.1 – Діаграма прецедентів системи

2.2 Діаграми класів (Class Diagram)

Для повного опису структури системи побудовано дві діаграми класів: (1) діаграма предметної області, що відображає ключові сутності та їх взаємозв'язки; (2) діаграма модульної структури, що показує розподіл функціональності між програмними модулями.

2.2.1 Діаграма класів предметної області

Основні сутності предметної області:

– **Location** — геокодована адреса з координатами та прив'язкою до вузла OSM-графа;

– **DeliveryStop** — точка доставки з адресою, координатами, вагою пакунку;

– **Vehicle** — транспортний засіб з типом (drive/bike/walk), місткістю;

– **VehicleRoute** — маршрут конкретного ТЗ: послідовність зупинок, повний шлях, загальний час;

– **Package** — фізичний пакунок з адресою, вагою, статусом доставки;

– **PackageDB** — SQLite-сховище пакунків;

– **UnreachableStop** — сутність, що описує недосяжні пункти.

Діаграма класів представлена на рисунку 2.2.

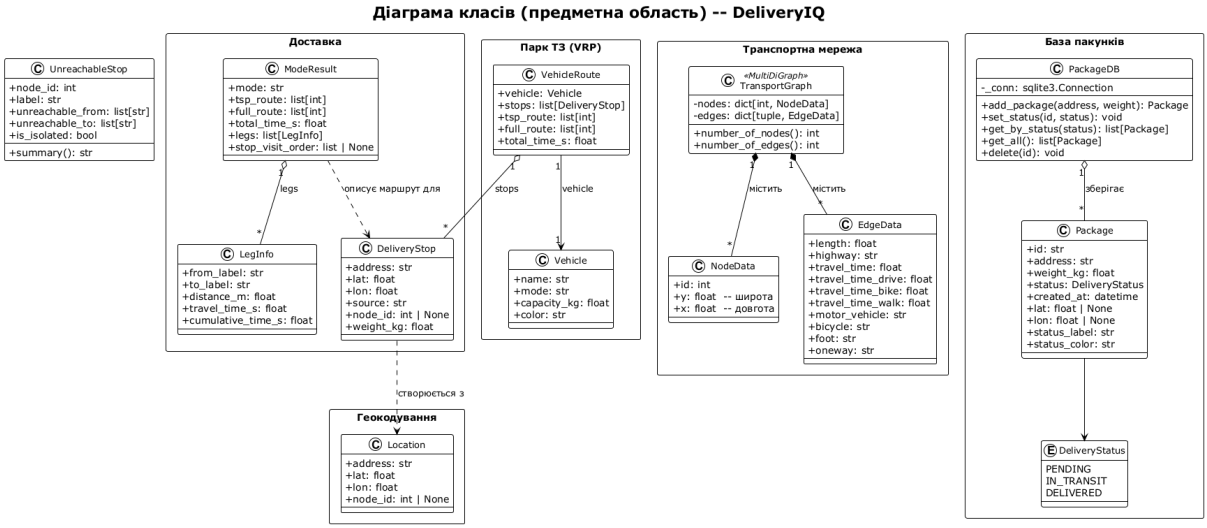

Рисунок 2.2 – Діаграма класів предметної області

2.2.2 Діаграма модульної структури

Система складається з шести основних модулів, кожен з яких реалізує окрему відповідальність:

– **app.py** — головний модуль Streamlit UI — координація всіх операцій, керування сесією, кешування, користувацький інтерфейс;

– **geocoder.py** — геокодування адрес через Nominatim API;

– **graph_builder.py** — завантаження OSM-мережі, побудова модальних графів з travel_time атрибутами;

– **route_solver.py** — побудова матриці відстаней (алгоритм Дейкстри), розв'язання транспортної задачі;

– **vrp_solver.py** — CVRP-оптимізація: розподіл зупинок між транспортними засобами, K-Means кластеризація, розв’язання TSP для кожного кластера;

– **visualizer.py** — побудова інтерактивної карти Folium з анімацією.

Модульна діаграма системи представлена на рисунку 2.3

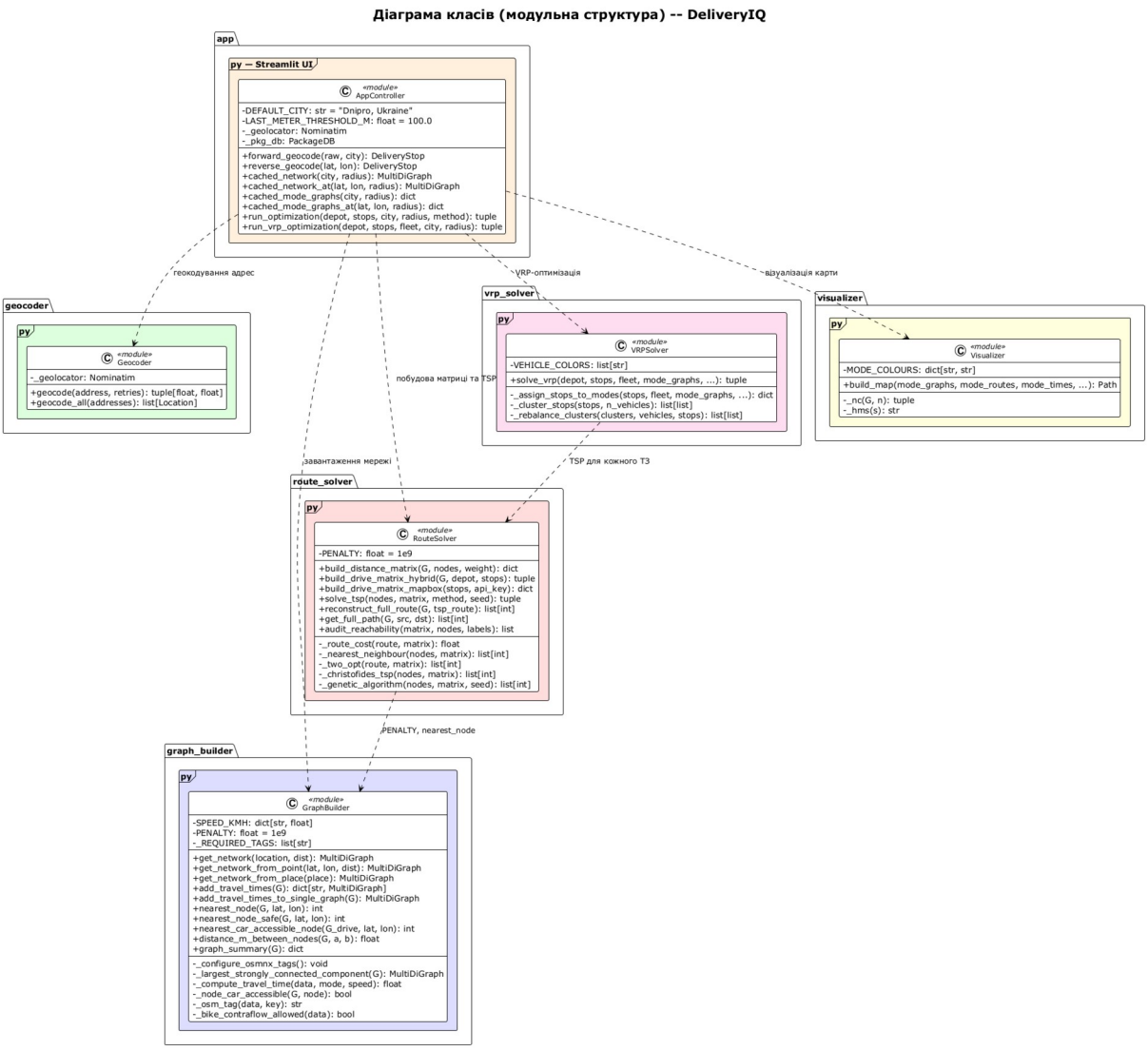

Рисунок 2.3 – Діаграма класів модульної структури

2.3 Діаграми послідовності (Sequence Diagram)

Діаграми послідовності описують динамічну взаємодію об'єктів системи у часі для ключових сценаріїв використання.

2.3.1 Процес оптимізації маршруту

Основний сценарій системи — повний цикл оптимізації маршруту — складається з шести послідовних фаз:

1) Завантаження OSM-мережі: graph_from_point → LSCC-оптимізація;
2) Прив'язка адрес до вузлів графа;
3) Побудова модальних графів (drive/bike/walk) з travel_time атрибутами;
4) Побудова матриці відстаней для кожного режиму;
5) Розв'язання TSP: автоматичний вибір методу за кількістю зупинок;
6) Візуалізація результатів на карті Folium.

Діаграма послідовності оптимізації маршруту вказана на рисунку 2.4.

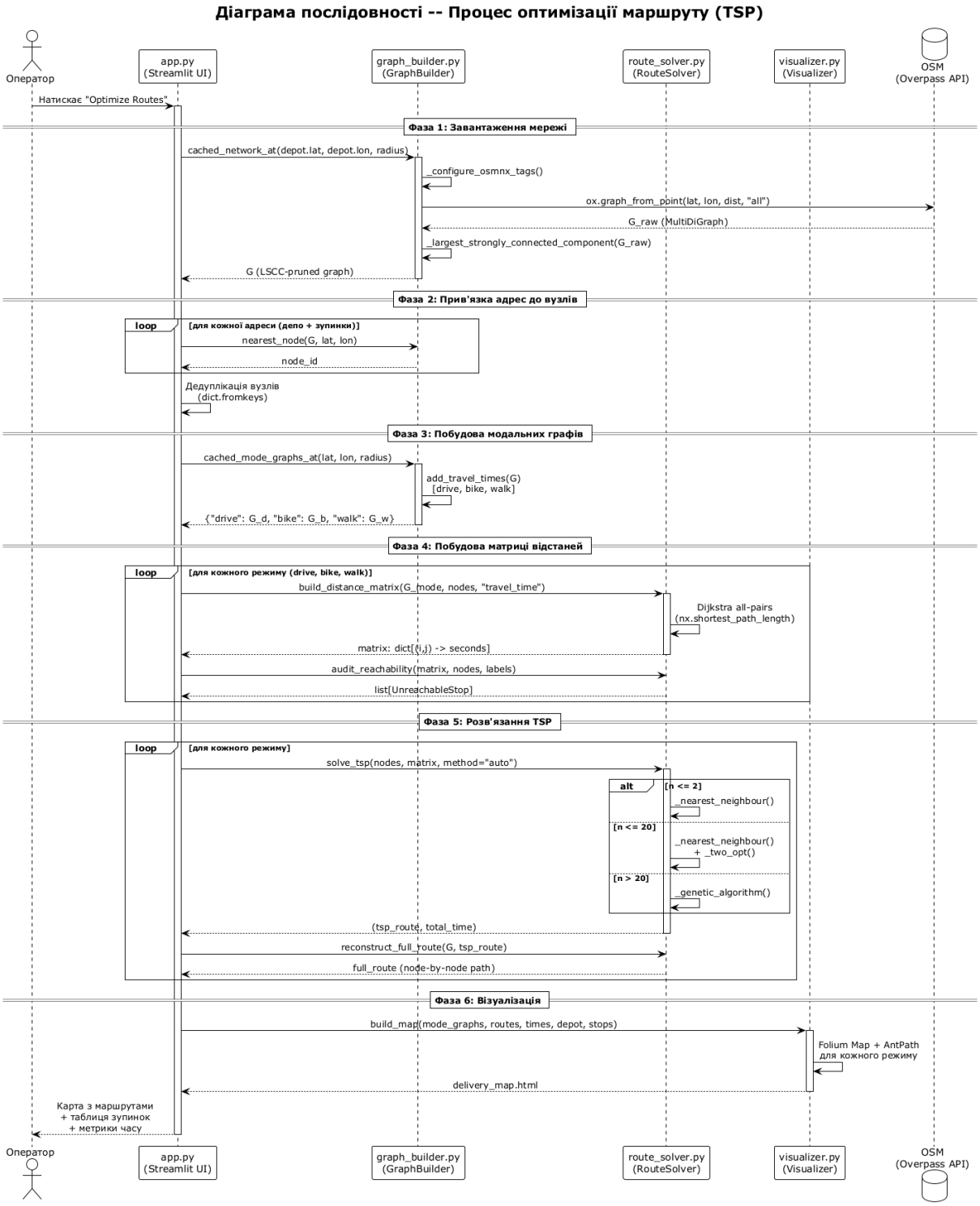

Рисунок 2.4 – Діаграма послідовності: оптимізація маршруту TSP

2.3.2 Геокодування адреси доставки

Діаграма описує два альтернативні сценарії додавання точки доставки: (1) введення текстової адреси та геокодуванням; (2) клік на карті з оберненим геокодуванням координат. Процес геокодування адреси доставки вказано на рисунку 2.5.

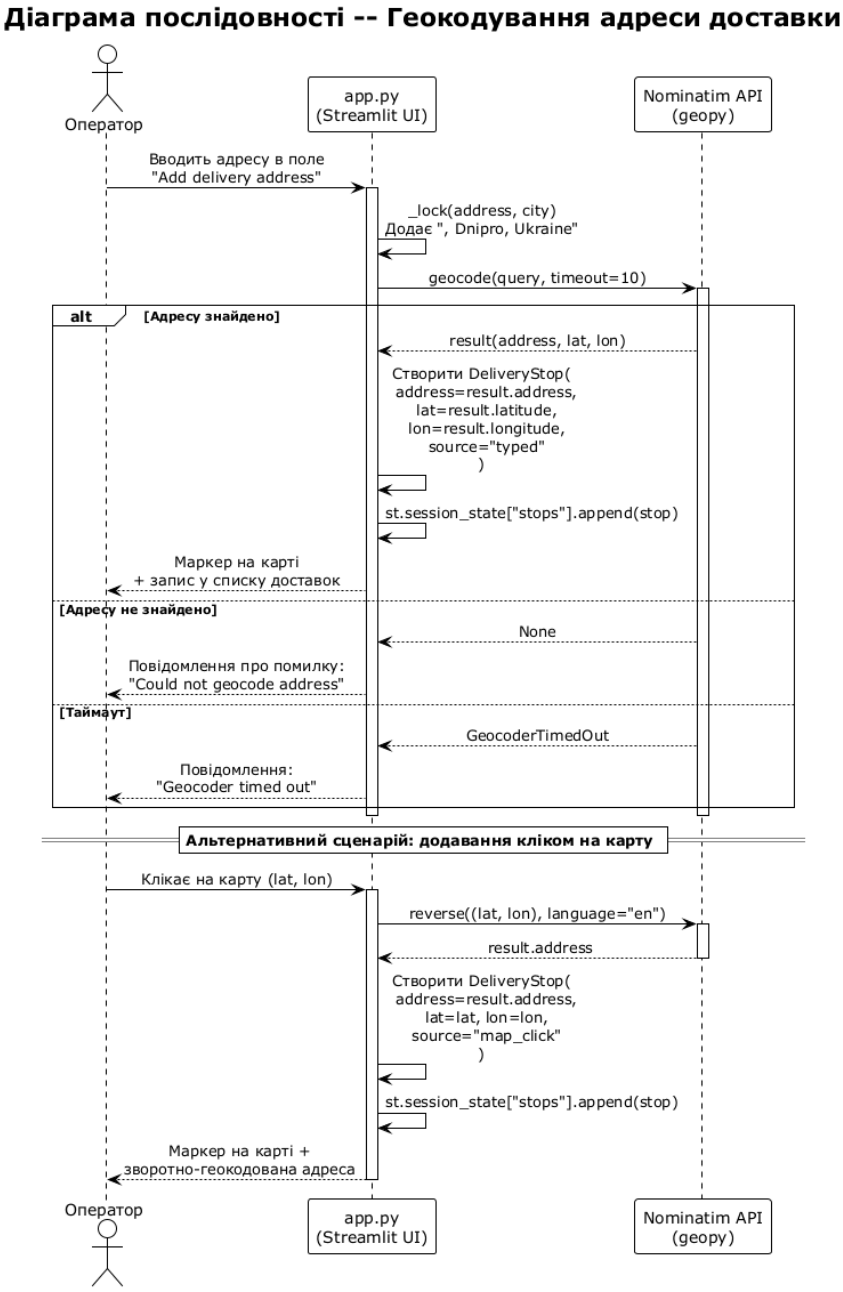

Рисунок 2.5 – Діаграма послідовності: геокодування адреси

2.3.3 Оптимізація VRP (декілька транспортних засобів)

Сценарій VRP-оптимізації включає три фази: (1) призначення зупинок до типів ТЗ за критерієм досяжності та залишкової місткості; (2) географічна кластеризація K-Means всередині кожного типу; (3) розв'язання задачі комівояжера для кожного транспортного засобу.

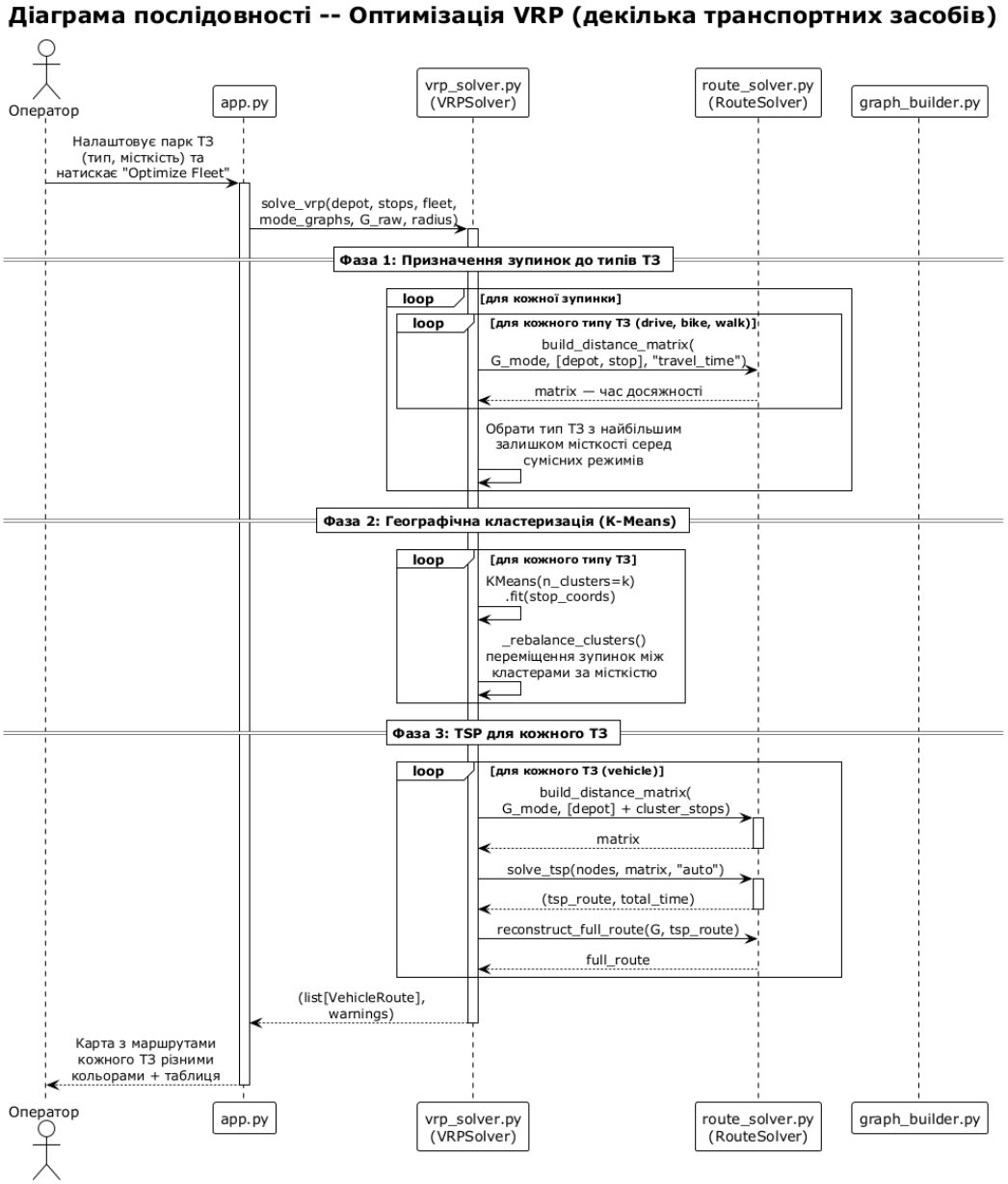

Рисунок 2.6 – Діаграма послідовності: VRP-оптимізація

2.4 Діаграма діяльності

Діаграма діяльності моделює послідовність дій під час виконання повного циклу оптимізації маршруту. Вона відображає:

– послідовні кроки обробки (завантаження графа, прив'язка, дедуплікація);

– умовні розгалуження (перевірка на порожній граф, недостатню кількість вузлів);

– паралельні потоки (побудова матриць та розв'язання TSP для трьох режимів одночасно);

– точки синхронізації.

Діаграма діяльності визначена на рисунку 2.7.

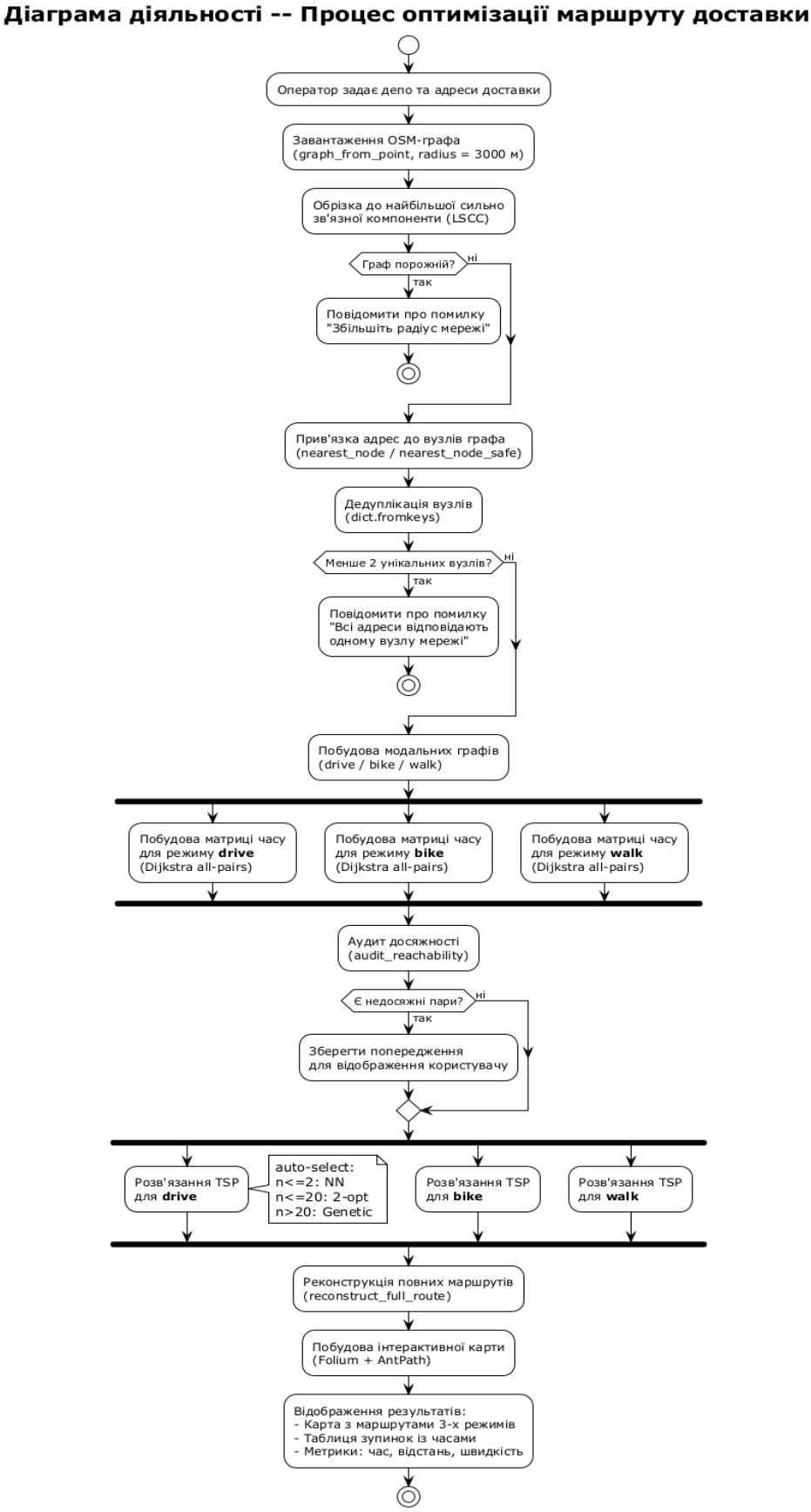

Рисунок 2.7 – Діаграма діяльності: процес оптимізації маршруту

2.5 Діаграма станів (State Diagram)

Діаграма станів описує життєвий цикл сутності Package (пакунок) у базі даних системи. Пакунок може перебувати в одному з трьох станів:

– **PENDING (Очікує)** – початковий стан після створення; адреса задана, координати ще не геокодовані;

– **IN_TRANSIT (В дорозі)** – пакунок включено в оптимізований маршрут ТЗ, координати геокодовані;

– **DELIVERED (Доставлено)** – пакунок успішно доставлено на адресу, маршрут завершено.

Переходи між станами ініціюються операціями set_status() класу PackageDB. Передбачено зворотний перехід IN_TRANSIT → PENDING у разі збою доставки.

Діаграма станів пакунків представлена на рисунку 2.8.

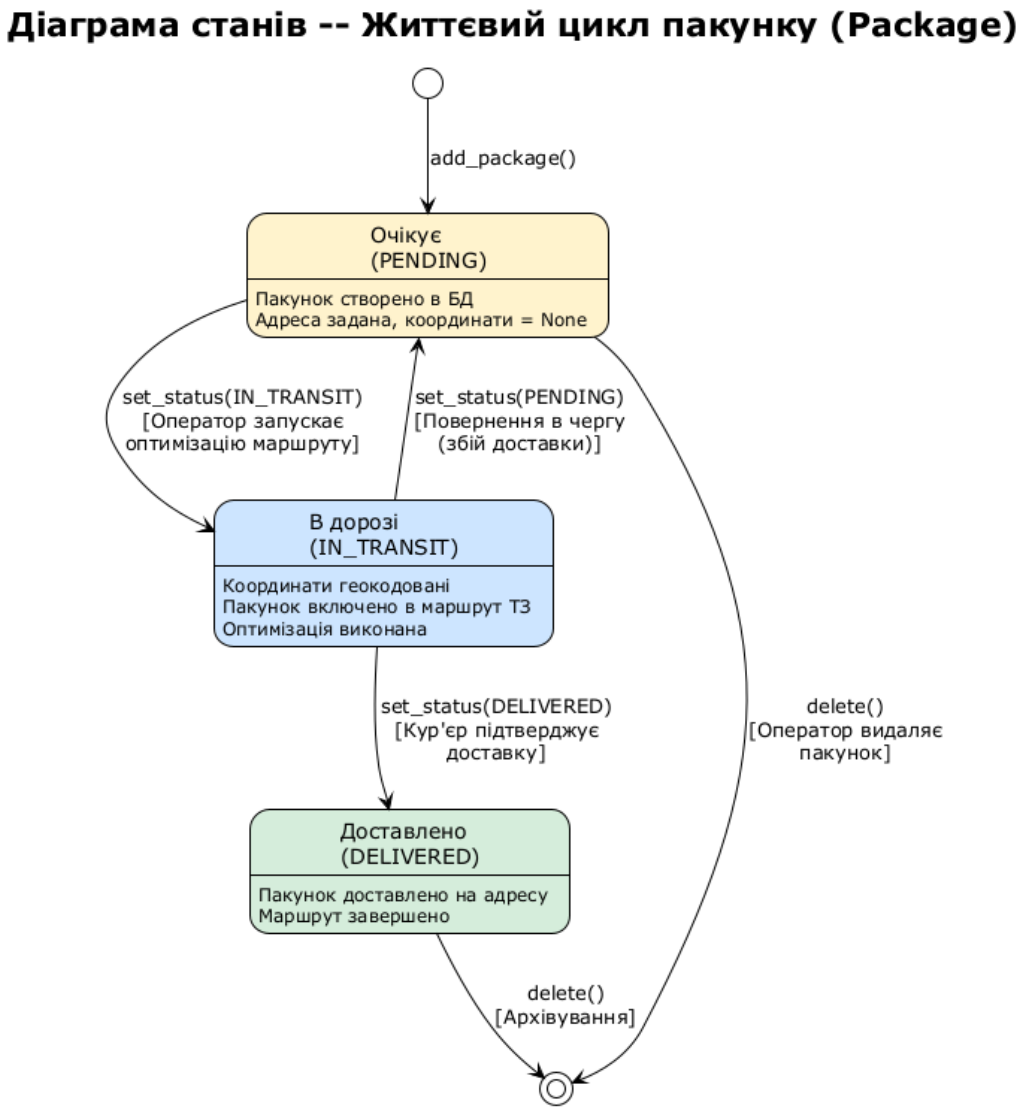

Рисунок 2.8 – Діаграма станів: життєвий цикл пакунку

2.6 Діаграма компонентів (Component Diagram)

Діаграма компонентів відображає фізичну структуру системи на рівні програмних модулів та їх залежностей. Система DeliveryIQ складається з:

– 6 основних Python-модулів (app.py, geocoder.py, graph_builder.py, route_solver.py, vrp_solver.py, visualizer.py);

– модуля package_db.py для роботи з SQLite базою даних packages.db;

– 6 зовнішніх Python-бібліотек (Streamlit, OSMnx, NetworkX, Folium, geopy, scikit-learn);

– 3 зовнішніх API-сервісів (Nominatim, Overpass, Mapbox).

Діаграма компонентів зображена на рисунку 2.9.

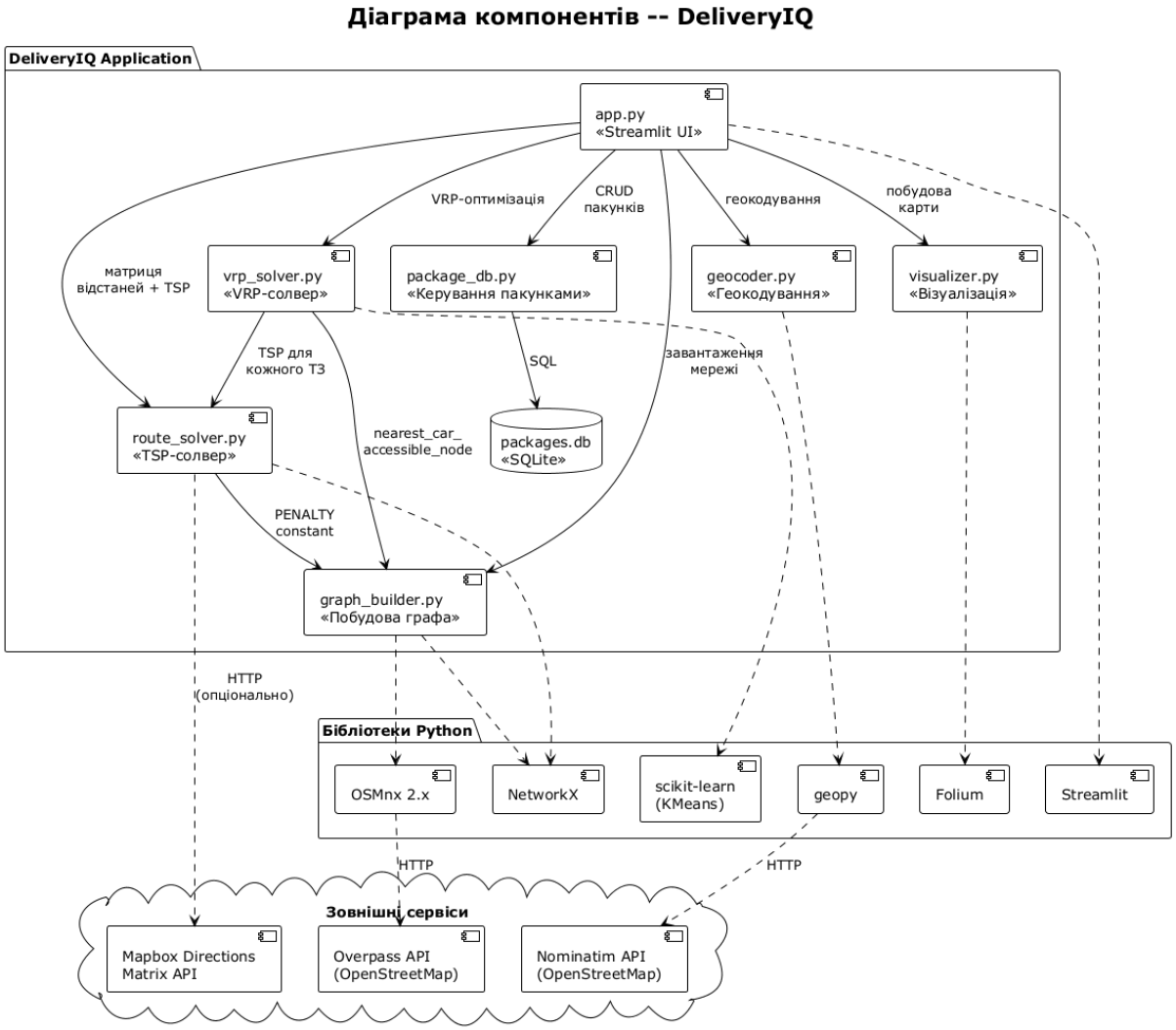

Рисунок 2.9 – Діаграма компонентів системи DeliveryIQ

2.7 Діаграма розгортання (Deployment Diagram)

Діаграма розгортання показує фізичну топологію розгортання системи:

– Клієнтський пристрій: веб-браузер (Chrome/Firefox), що взаємодіє зі Streamlit-сервером через HTTP/WebSocket;

– Сервер додатку: Python 3.11+ Runtime із Streamlit Server (порт 8501), усіма модулями системи, SQLite базою даних та кешем GraphML файлів;

– Зовнішні хмарні сервіси: OpenStreetMap (Nominatim + Overpass API) та Mapbox.

Кешування OSM-графів у форматі GraphML дозволяє уникнути повторних звернень до Overpass API та значно пришвидшує повторні запуски оптимізації.

Діаграма, що описує процес розгортання системи представлена на рисунку 2.10.

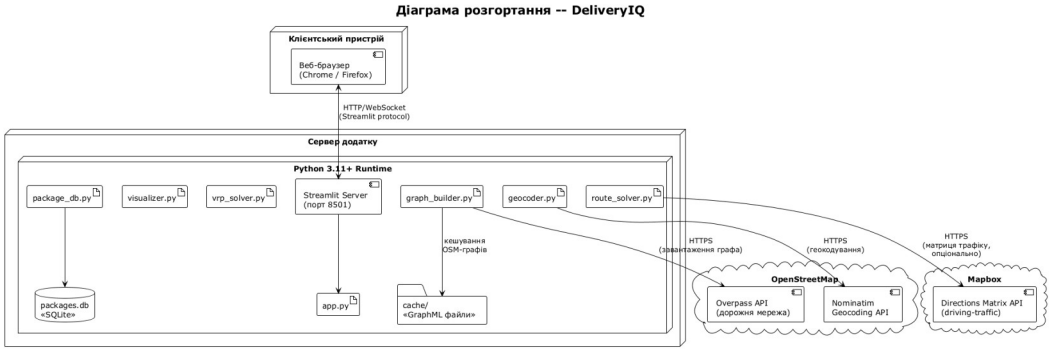

Рисунок 2.10 – Діаграма розгортання системи DeliveryIQ

3 РОЗРАХУНКОВИЙ РОЗДІЛ

3.1 Вибір засобів розробки

Для реалізації системи обрано наступний технологічний стек:

- Python 3.11+ – основна мова програмування;
- Streamlit – фреймворк для побудови інтерактивного веб-інтерфейсу;
- OSMnx 2.x – бібліотека для завантаження та аналізу мереж OpenStreetMap;
- NetworkX – бібліотека для роботи з графами (Dijkstra, побудова маршрутів);
- Folium та AntPath – інтерактивна картографічна візуалізація з анімованими маршрутами;
- geopy (Nominatim) – геокодування адрес у координати;
- scikit-learn (KMeans) – географічна кластеризація для VRP;
- SQLite – легка реляційна БД для зберігання пакунків;
- PlantUML – засіб побудови UML-діаграм для проєктування.

3.2 Структура програмного забезпечення

Архітектура системи побудована за принципом чіткого розділення відповідальностей. Модулі для обробки даних:

- app.py (Streamlit UI) — точка входу, координація всіх операцій;
- geocoder.py — перетворення адрес у координати (Nominatim);
- graph_builder.py — завантаження OSM-мережі, LSCC-обрізка, модальні графи;
- route_solver.py — матриця відстаней (Dijkstra) + TSP (NN / 2-opt / GA / Christofides);
- vrp_solver.py — CVRP: розподіл зупинок, K-Means, TSP для кожного ТЗ;
- visualizer.py — Folium-карта з AntPath-маршрутами;
- package_db.py — SQLite CRUD для пакунків.

3.3 Ключові алгоритми

Система реалізує декілька алгоритмів оптимізації маршрутів:

– Nearest Neighbour (NN) – жадібний алгоритм $O(n^2)$, використовується для 1–2 зупинок;

– 2-opt – ітеративне покращення NN-розв'язку шляхом обміну ребер, $O(n^3)$ за ітерацію;

– Генетичний алгоритм (GA) – Order Crossover (OX) з елітизмом, 120 особин та 400 поколінь, для задач з великою кількістю зупинок;

Автоматичний вибір алгоритму (method="auto") визначається кількістю зупинок: $n \leq 2 \rightarrow$ NN, $3 \leq n \leq 20 \rightarrow$ 2-opt, $n > 20 \rightarrow$ генетичний алгоритм. Це забезпечує баланс між якістю розв'язку та часом обчислення.

Механізм «останньої милі» для автомобільного режиму: якщо адреса доставки знаходиться на пішохідній вулиці, автомобіль паркується на найближчому вузлі, а залишок шляху (до 100 м) долається пішки. Час переїзду включає обидві частини.

3.4 Аналіз отриманих результатів

У процесі проєктування інформаційної системи DeliveryIQ було розроблено комплект із десяти UML-діаграм, що забезпечують повне покриття всіх аспектів системи відповідно до об'єктно-орієнтованого підходу. Аналіз діаграми прецедентів (рис. 2.1) підтверджує коректність визначення функціональних вимог системи.

Виявлено одного основного актора — Оператора логістики — та трьох зовнішніх акторів-систем: Nominatim API, OpenStreetMap (Overpass API) та Mapbox API. Визначено дев'ять основних прецедентів, між якими встановлено відношення «include» та «extend», що відображає реальну залежність операцій: зокрема, запуск оптимізації обов'язково включає завантаження дорожньої мережі та побудову матриці відстаней.

Аналіз діаграм класів (рис. 2.2, 2.3) показав, що предметна область коректно декомпозована на сім ключових сутностей: Location, DeliveryStop, Vehicle, VehicleRoute, Package, PackageDB, UnreachableStop. Між сутностями встановлено відношення агрегації та залежності, що відповідає принципам слабкого зв'язування (loose coupling). Діаграма модульної структури демонструє чіткий розподіл відповідальностей між шістьма модулями системи відповідно до принципу єдиної відповідальності (Single Responsibility Principle).

Аналіз діаграм послідовності (рис. 2.4–2.6) дозволив верифікувати коректність взаємодії між компонентами системи у трьох ключових сценаріях. Виявлено, що процес оптимізації маршруту складається з шести послідовних фаз, причому фаза побудови матриць відстаней виконується паралельно для трьох транспортних режимів, що забезпечує скорочення загального часу обчислення. Механізм автоматичного вибору алгоритму TSP (NN → 2-opt → GA залежно від кількості зупинок) забезпечує баланс між якістю розв'язку та обчислювальною складністю.

Аналіз діаграми діяльності (рис. 2.7) підтвердив наявність усіх необхідних умовних розгалужень та точок синхронізації. Зокрема, виявлено два критичні захисні умови: перевірку на порожній граф після завантаження OSM-мережі та перевірку на недостатню кількість унікальних вузлів після дедуплікації, що запобігає некоректному запуску алгоритмів оптимізації.

Аналіз діаграми станів (рис. 2.8) показав повноту моделювання життєвого циклу сутності Package. Визначено три стани (PENDING, IN_TRANSIT, DELIVERED) та всі допустимі переходи між ними, включаючи зворотний перехід IN_TRANSIT → PENDING для обробки збоїв доставки. Кількість станів є мінімально необхідною для коректного відображення бізнес-логіки системи.

Аналіз діаграм компонентів та розгортання (рис. 2.9, 2.10) підтвердив архітектурну обґрунтованість рішення. Система складається з семи Python-модулів із чітко визначеними інтерфейсами взаємодії та трьох рівнів: клієнтського пристрою, сервера додатку та зовнішніх хмарних сервісів. Кешування OSM-графів у форматі GraphML усуває надлишкові звернення до Overpass API, що є критично важливим з урахуванням обмежень rate limiting зовнішнього сервісу.

Таким чином, розроблений комплект UML-діаграм повністю описує статичну структуру, динамічну поведінку та фізичну архітектуру системи DeliveryIQ. Проєктна документація є достатньою основою для подальшої реалізації, тестування та супроводу системи.

ВИСНОВКИ

В результаті курсової роботи було спроєктовано інформаційну систему «DeliveryIQ» для оптимізації розподілу вантажів у транспортній мережі з використанням об'єктно-орієнтованого підходу.

В ході роботи було:

1) проведено аналіз предметної області маршрутизації транспортних засобів та формалізовано задачу CVRP;

2) побудовано діаграму прецедентів, що визначає функціональні вимоги системи та її акторів;

3) розроблено дві діаграми класів: предметної області (ключові сутності та зв'язки) та модульної структури (розподіл функціональності між модулями);

4) створено три діаграми послідовності для основних сценаріїв: оптимізація TSP, геокодування адрес, VRP-оптимізація;

5) побудовано діаграму діяльності процесу оптимізації з паралельними потоками;

6) розроблено діаграму станів для життєвого циклу пакунку;

7) побудовано діаграми компонентів та розгортання, що описують фізичну архітектуру системи;

8) описано технологічний стек та ключові алгоритми оптимізації.

Загалом побудовано 10 UML-діаграм, що забезпечують повне покриття статичної структури, динамічної поведінки та фізичної архітектури системи DeliveryIQ. Розроблений комплект проєктної документації може бути використаний як основа для подальшої реалізації, тестування та супроводу системи.

Пояснювальну записку до курсової роботи оформлено у відповідності до методичних рекомендацій до змісту та структури курсової роботи з дисципліни

«Проєктування інформаційних систем» за освітнім рівнем «бакалавр» для студентів спеціальності 122 Комп'ютерні науки [10].

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

1. Шаховська Н. Б., Литвин В. В. Проектування інформаційних систем : навч. посіб. / за ред. В. В. Пасічника. Львів : Магнолія 2006, 2017. 380 с.

2. Проектування інформаційних систем : посіб. / за ред. В. С. Пономаренка. Київ : Академія, 2002. 488 с.

3. Гайна Г. А. Основи проектування баз даних : навч. посіб. Київ : Кондор, 2008. 200 с.

4. Зеленцов Д. Г., Ляшенко О. А., Науменко Н. Ю. Информационное обеспечение расчетов корродирующих объектов. Математические модели и концепция проектирования систем : монография. Днепропетровск : УГХТУ, 2012. 264 с.

5. Авраменко В. С., Авраменко А. С. Проектування інформаційних систем : навч. посіб. Черкаси : Черкаський національний університет ім. Б. Хмельницького, 2017. 434 с.

6. Boeing G. OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*. 2017. Vol. 65. P. 126–139.

7. Toth P., Vigo D. The Vehicle Routing Problem. Philadelphia : SIAM, 2002. 385 c. (Monographs on Discrete Mathematics and Applications).

8. Christofides N. Worst-case analysis of a new heuristic for the travelling salesman problem : Report 388. Pittsburgh : Carnegie Mellon University, 1976. 13 p.

9. Goldberg D. E. Genetic Algorithms in Search, Optimization, and Machine Learning. Reading : Addison-Wesley, 1989. 412 p.

10. Методичні вказівки до виконання курсових робіт з дисципліни «Проектування інформаційних систем» за освітнім рівнем «бакалавр» для студентів спеціальності 122 «Комп'ютерні науки» / уклад. О.А. Ляшенко. – Дніпро: ДВНЗ «УДХТУ», 2020. – 56 с.